Aggressive vs. Conservative TTG

What was prefetched, what was needed, and what the hit-rate denominator includes

JDrive

Aggressive: 832 confirmed avoidable full-node fetches.
Conservative: none found in the two audited classes.

LittleX

357 prefetched and used in both policies.
Access-only reads are now covered by Conservative.

RecipePantry

Coverage remains lower: 31.10% vs. 67.07%.
Unknown origins / later checkpoints are still gaps.

24 fresh workflows • 3 apps • n = 3 per policy • GIL-off threads

Conservative includes the new Access-demand implementation.
“832” is a confirmed lower bound, not an exhaustive minimal-read oracle.

Define “unnecessary” at the payload boundary

A node need not be visited to be useful; a node can be accessed and still be avoidably fetched.

Necessary payload

Fields are needed for the result or later computation. An ordinary helper read counts, even without Visit.

Filter-only payload

Full node fetched only to reject it using a predicate. SQL can evaluate the filter without returning the node.

Unused prefetch

Payload returned by the prefetch worker, but never read by foreground code. This is also avoidable.

Confirmed unnecessary prefetch = unused + disjoint filter-only payloads

The database still needs to evaluate predicates. We count complete node payloads transferred / materialized at the client, not PostgreSQL rows scanned or bytes read from disk.

A zero in the confirmed-count table means “none found in these two classes”; it is not a proof that all remaining full payloads or properties are semantically necessary.

A reproducible comparison, not a new workload

Same fixed business operations and current runtime in both policy configurations

App	Measured workflow	Policy pairs
JDrive	List / visible tree / trash / create folder	3 measured + 1 warmup
LittleX	Feeds / likes / follow / profile / create tweet	3 measured + 1 warmup
RecipePantry	Plan / buy / cook / boards / shopping states	3 measured + 1 warmup

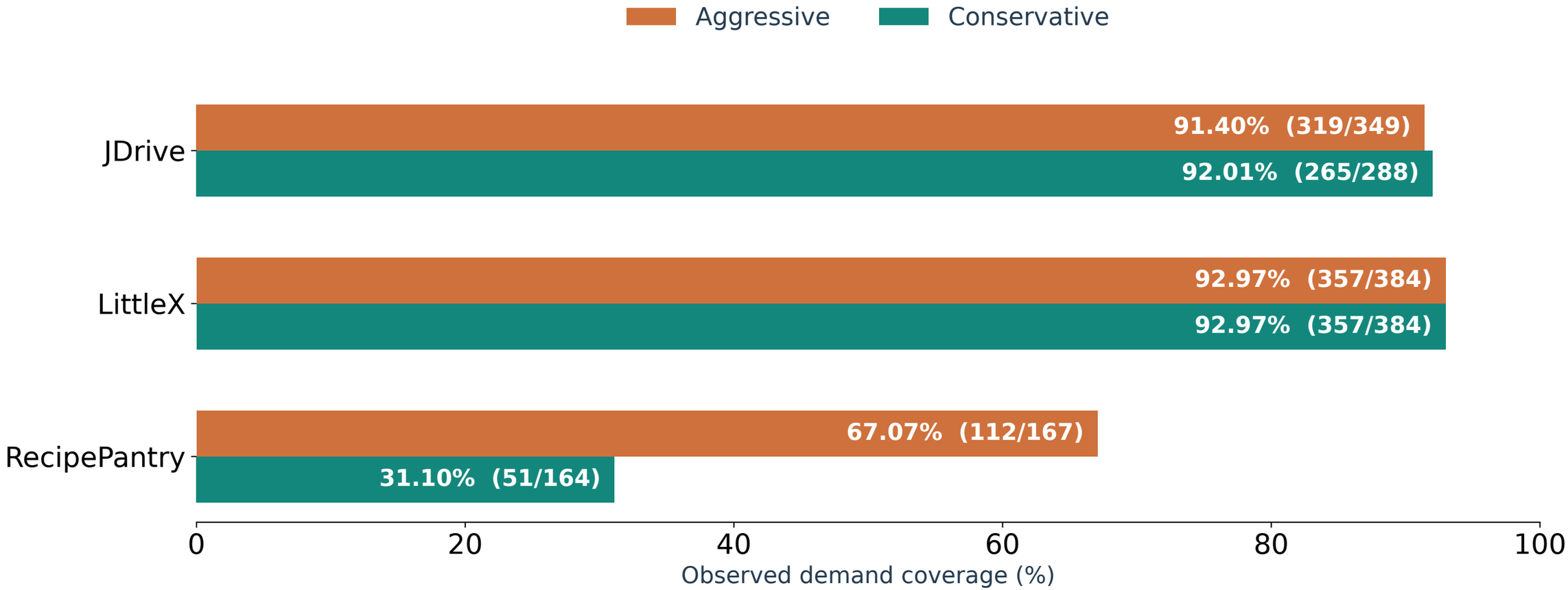
Thread executor; GIL disabled

1 worker, limit 2048; actual execution assertions retained. All business/state checks pass.

6 excluded warmups + 18 measured workflows = 24 captures. Counts are identical across n=3.
Unit: distinct node IDs per request/session, summed over the workflow. IDs may repeat across requests.
Both policies share the new frozen runtime; “Aggressive” is the legacy policy, not an old binary.

Hit rate = observed demand coverage

H = prefetched IDs later used / all foreground-demand IDs; root reads are included here



Demand means successful foreground Session.get / batch_get, including memory hits.
This is not a PostgreSQL cache-hit rate or a proven minimum-semantic-demand denominator.

Payload counts: use is not the same as necessity

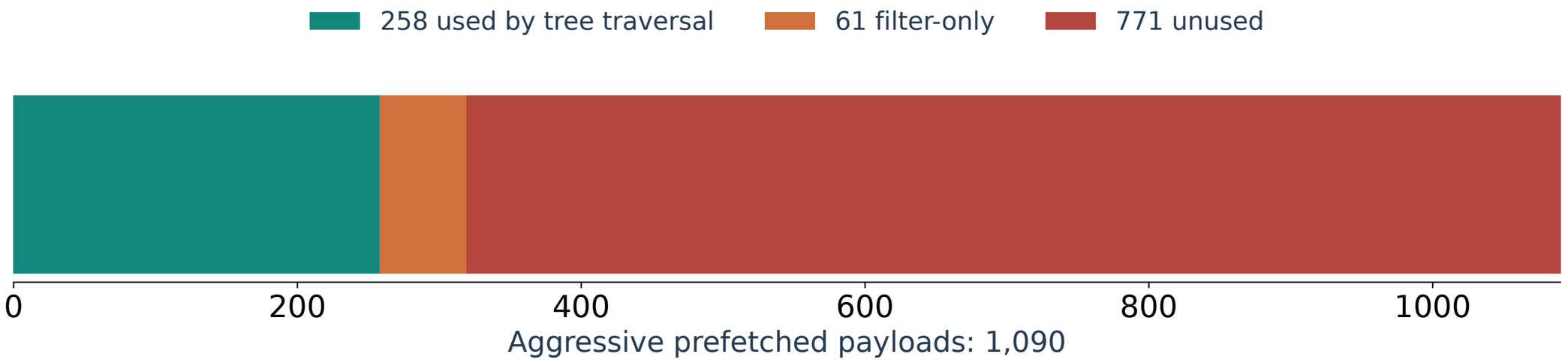
All numbers are per workflow; n=3 produces identical counts

App / policy	Prefetched	Used	Unused	Filter-only	Confirmed unnecessary
JDrive / Agg.	1090	319	771	61	832 (at least)
JDrive / Cons.	265	265	0	0	0 found
LittleX / Agg.	357	357	0	0	0 found
LittleX / Cons.	357	357	0	0	0 found
RecipePantry / Agg.	112	112	0	3	3 (at least)
RecipePantry / Cons.	51	51	0	0	0 found

Unused and filter-only sets are disjoint. Filter-only counts were checked against paired request IDs, source predicates, actual node types and state. All zero counts refer only to these audited classes.

JDrive: 771 unused is not the whole story

The predicate trashed == False avoids another 61 full-node fetches per workflow



Aggressive: at least 832

771 unused + 61 read only as rejected trashed candidates; full fetch avoidable.

Conservative: 265 used

258 tree nodes + 7 ordinary ListFolder Accesses. No examples in either class.

Matched outputs and final states. SQL still evaluates trashed; the claim is avoiding full-node payloads.

LittleX: Access-only demand is real demand

Conservative now matches Aggressive: 357 prefetched, 357 used, 0 unused

Conservative	Before Access support	After Access support
get_profile	1 used prefetch	70 used prefetches
follow_user	0	1
Whole workflow	287	357

357 = 287 prefetched-and-entered + 70 useful without Visit. Not entering a node is not evidence of waste.
Before/after counts are historical same-workflow observations; no cross-run latency comparison.

RecipePantry: the denominator and the coverage gap

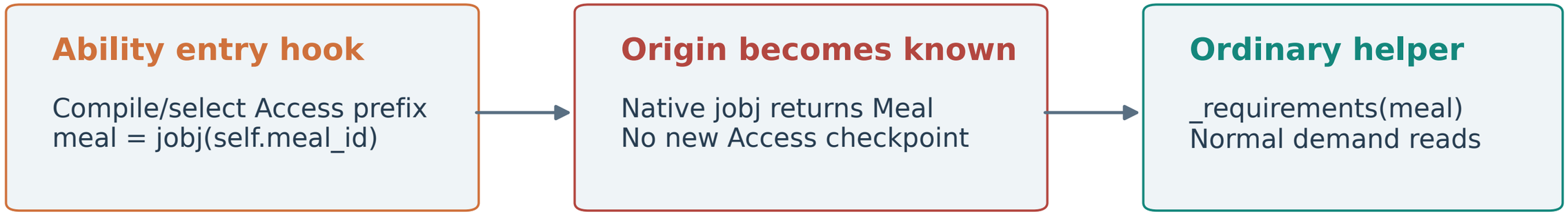
Root reads explain part of the low percentage; the remaining gap is real

Measure	Aggressive	Conservative
All foreground demand	167	164
Root demand (global + user)	39	39
Used prefetch	112	51
All-demand coverage	67.07%	31.10%
Non-root observed coverage	112 / 128 = 87.50%	51 / 125 = 40.80%

The root-excluded view changes only the denominator; it does not erase business misses. Aggressive has 3 confirmed filter-only cooked Meal prefetches in ShoppingList, despite zero unused prefetches. Conservative still has substantially lower coverage, not a universal precision advantage.

Why Conservative misses later Accesses

Current implementation gap: unresolved origin ends the executable prefix



Meal → Recipe → Requirement → Ingredient

CookMeal has no later explicit graph query after this helper

It checks stock and writes inventory / Consumption edges. The queries are inside the helper.

PlanMeal / CookMeal / BuyIngredient have no later prefetch submissions in these captures.
A later ability entry can trigger prefetch, but an ordinary helper call is not a new ability entry.

SQL counts are a separate outcome

These precision captures do not establish an uninstrumented latency speedup

App	Aggressive data / control	Conservative data / control
JDrive	208 / 52	53 / 54
LittleX	74 / 70	73 / 71
RecipePantry	189 / 150	207 / 144

JDrive combines pruning with fewer data queries

LittleX has almost equal SQL counts. RecipePantry still uses more data queries in Conservative.

A Session demand is not necessarily a new SQL call. These are separate measurements.
Additional Access paths reuse the existing resolver; they are not all fused into one SQL statement.

What to improve next

Recover useful coverage without replacing selective analysis with broad traversal

1 Model direct identity lookup as an Access origin

jobj(id) → object identity → typed subsequent graph paths.

2 Resume after the last unresolved dependency

Insert a checkpoint after meal becomes known; preserve effects and native execution order.

3 Extend the necessity audit

Separate payload, predicate columns and identity-only reads; keep root/entry demands visible.

4 Re-test both objectives

Report useful coverage and confirmed unnecessary fetches together; keep slow/unsupported cases.

Evidence and scope

All plotted figures come from immutable accepted captures or explicitly labelled historical counts

24 captures / 18 measured / 6 excluded warmups

Native responses, persisted state, restarts, fresh sessions and GIL-off checks passed.

Raw precision audit

1,920 sealed files rehashed; all measured counts identical across three repetitions.

Editable and reproducible

slide-data.json + derive.py + render.py; accepted results.json / table.csv and raw case archives.

Definitions: “hit rate” here is observed demand coverage. “Unnecessary” is avoidable full-node payload, not “not visited.” Confirmed unnecessary counts are lower bounds, not complete semantic-minimality proofs. No DB experiment was added to produce this presentation.